ZentroMart VoiceOps

VoiceOps 2026 – Capstone Project Report

Submitted by: Thalla Hruthikesh

1. Project Overview

ZentroMart is a fictional e-commerce store provided as part of the VoiceOps 2026 capstone.

The objective of this project was to develop an AI-powered customer support assistant that can understand customer queries, retrieve information from store documents, handle follow-up conversations and assist with order-related requests.

I developed ZentroMart VoiceOps using Python, a large language model, a retrieval pipeline and the speech services provided by Gnani. The application supports both text and voice interactions through a Streamlit interface.

Rather than building a chatbot that only generates responses, the aim was to develop a system that uses available store information, maintains conversation context and performs specific support operations when permitted.

2. Implementation

The project was developed in four stages.

Stage 1: LLM-Based Customer Support

The first version uses Groq with the `openai/gpt-oss-120b` model.

It follows system instructions to define the assistant's role, response behaviour and limitations. This version answers customer questions without retrieving external documents or accessing business tools.

It serves as the baseline for comparing the improvements introduced by retrieval.

Stage 2: Retrieval-Augmented Generation (RAG)

I connected the assistant to the supplied store knowledge base to make its responses more relevant to the available information.

The retrieval pipeline includes:

- **Document processing:** Markdown documents are divided into sections while retaining document IDs, types, status and effective dates.
- **Multilingual embeddings:** A MiniLM sentence-transformer model generates embeddings for document sections.
- **Semantic retrieval:** A NumPy-based cosine similarity index identifies relevant sections.
- **Keyword matching:** An additional keyword score helps retrieve exact terms and product identifiers.
- **Product retrieval:** Product records are indexed alongside policy documents.
- **Policy precedence:** When an older policy is retrieved, its current successor is also included in the context.

The retrieved information is passed to the language model before it generates an answer. The application also records retrieved chunks and cited document IDs for inspection.

Stage 3: Conversation Memory and Business Tools

The assistant was extended to support multi-turn customer conversations.

It maintains recent conversation messages, the active order and verified order information within a session.

Python-controlled functions handle the following operations:

- Looking up synthetic order details.
- Verifying customer information before allowing access to an order.
- Enforcing a maximum of two verification attempts per order.
- Removing delivery pincodes from successful lookup results before passing order details to the language model.
- Creating local synthetic support tickets for unresolved customer issues.

The assistant does not directly execute refunds or cancellations. These actions are outside the capabilities of the supplied tools.

The dataset reference date is **20 October 2026**. Business-day calculations exclude Sundays and the holidays supplied in the dataset.

Stage 4: Voice Integration

Gnani speech-to-text (STT) and text-to-speech (TTS) were integrated to support voice-based customer interactions.

The workflow is:

Customer Speech → Speech Recognition → AI Agent → Response Generation → Speech Synthesis → Spoken Response

The application accepts microphone recordings, converts speech into text and sends the transcript to the same customer support agent used for typed questions.

The generated response is then converted into speech using Gnani TTS.

The configured defaults are English, Kaveri and timbre-v2.5. Long PCM WAV recordings are split into clips of up to 25 seconds before transcription.

I also developed a batch audio runner that processes multiple recordings and saves response text, generated audio and JSON execution details. Individual file failures are recorded in a manifest.

I manually tested the complete microphone-to-spoken-response workflow successfully. However, formal speech-quality testing across different speakers, accents and noisy environments has not been conducted.

3. Technology Stack

Component	Technology
Programming Language	Python
User Interface	Streamlit
Language Model	Groq – openai/gpt-oss-120b
Embeddings	Multilingual MiniLM
Retrieval	NumPy, Cosine Similarity and Keyword Matching
Speech-to-Text	Gnani STT
Text-to-Speech	Gnani TTS
Testing	Python unittest

4. Evaluation and Results

I evaluated the system using the supplied practice questions and multi-turn conversation dataset.

The evaluation covers three modes: the LLM baseline, RAG and multi-turn conversations.

4.1 Baseline and RAG Evaluation

The original baseline and RAG evaluation results are:

Evaluation Mode	Total	Completed	Proxy Passes	Decision Correct	Errors
LLM Baseline	52	52	4	7	0
RAG	52	52	13	24	0

On the same 52-question evaluation set, RAG recorded 13 proxy passes compared with 4 for the baseline.

This represents nine additional proxy passes under the same evaluation rules. The result suggests that retrieving relevant store information improved performance on the supplied questions.

The saved RAG evaluation also reported a mean retrieved-evidence recall of **0.518** across questions with gold evidence.

This metric measures how much of the expected evidence was retrieved. It does not establish whether the generated answer interpreted or used that evidence correctly.

4.2 Multi-Turn Conversation Evaluation

The initial conversation evaluation was interrupted when the Groq daily token quota was exhausted. At that stage, 24 of the 96 turns had completed.

I subsequently improved the evaluation script to support checkpointing and resuming. This allowed the evaluation to continue from saved progress instead of restarting completed conversations.

The final resumed evaluation completed all 96 turns.

Metric	Final Result
Total Conversation Turns	96
Completed Turns	96
Pending Turns	0
Proxy Passes	47
Decision-Correct Flags	50

The final evaluation covered the supplied conversation cases from PC002 to PC024.

The original evaluation files were preserved, and the resumed results were saved separately.

4.3 Understanding the Evaluation Metrics

The proxy-pass metric checks whether an answer contains expected information, makes an acceptable decision and avoids specified forbidden phrases.

Decision-correct flags measure a separate aspect of the response.

These are diagnostic metrics generated by the evaluation script, not official accuracy scores. Literal matching may fail to recognise correct paraphrases or multilingual equivalents. Similarly, passing a proxy check does not guarantee that an answer is completely correct or safe.

For this reason, I have reported the actual counts rather than presenting them as overall model accuracy.

5. Observations and Challenges

The evaluation helped identify some weaknesses in the assistant.

One recurring issue was unnecessary clarification.

For example, in practice question PQ0002, the RAG implementation identified the Assured badge but asked for order verification instead of providing all the required public warranty information. The baseline also requested an order ID.

The saved evaluation showed the following response patterns:

Response Type	Baseline	RAG
Clarifications	39	29
Answers	2	22

These counts are from the original practice-question evaluation.

The results suggest that retrieval helped the assistant provide more direct answers. However, it still sometimes treats general product or policy questions as private order requests.

This is an area I would improve by separating public-information queries from requests that actually require customer verification.

Another challenge was maintaining conversation context while respecting verification restrictions. The evaluation work also highlighted the importance of saving intermediate state so that interrupted tool operations are not repeated during a resumed run.

6. Testing and Final Improvements

The project includes automated offline tests for important application behaviours.

18 offline unit tests passed, including checks for:

- Customer verification and retry limits.
- Conversation session handling.
- Business-day calculations.
- Document metadata and retrieval behaviour.
- Sensitive information redaction.
- Support ticket creation.
- Citation filtering.
- Empty audio transcript handling.
- Evaluation checkpoint and resume behaviour.

I also made several improvements during final preparation:

- Added evaluation modes for baseline, RAG and conversations.
- Implemented append-only evaluation results and checkpoint/resume support.
- Added handling for daily quota exhaustion and other evaluation errors.
- Added state restoration for completed conversation turns.
- Improved recovery of interrupted tool operations to avoid repeating completed ticket creation.
- Corrected the Plus-member refund wording: refunds are initiated at pickup, rather than described as processed or settled at pickup.
- Improved provider error handling so raw API error details are not exposed unnecessarily.

- Added handling for empty speech transcripts through a repeat request.

The offline tests use scripted responses and mocked HTTP calls. They verify application logic but should not be confused with live LLM evaluation.

Python compilation, imports, dependency checks and official dataset validation were also completed during project verification.

7. Limitations and Future Improvements

There are still areas where the system can improve.

Retrieval and response quality: Retrieving the correct document does not always guarantee that the model interprets it correctly. Better evidence selection and stricter answer validation would help.

Unnecessary clarification: The assistant sometimes asks for order verification when a customer is only asking about public product or policy information. Improving intent routing would make interactions more natural.

Evaluation: The current evaluation uses a synthetic dataset and diagnostic scoring. A human-reviewed test set would provide a better assessment of correctness, completeness and grounded responses.

Conversation handling: Although all 96 conversation turns were processed, the proxy results show that completing the conversation flow does not necessarily mean every response is correct.

Voice experience: The speech pipeline was manually tested, but it has not been formally evaluated across different accents, microphones and background-noise conditions.

Security: The order verification process is an educational simulation using synthetic records. It is not intended to replace production-grade customer authentication.

Some useful next steps would be to improve public-query versus private-order routing, review the conversation responses that did not pass the proxy checks, strengthen evidence validation and conduct broader voice testing.

8. Conclusion

Through this project, I developed a customer support assistant that combines language models, document retrieval, conversation memory, business tools and voice interaction.

The implementation helped me understand how these components work together in an applied AI system, particularly the challenges of grounding responses, managing multi-turn conversations and controlling access to customer information.

The evaluation showed an improvement in proxy passes when retrieval was introduced, and the final resumed conversation evaluation successfully processed all 96 turns.

At the same time, the results highlighted areas that still need improvement, especially unnecessary clarification and response correctness.

The project gave me practical experience in building and evaluating an end-to-end AI application rather than focusing only on generating responses from a language model.

9. Project Resources

Source Code: https://github.com/Hruthikesh/Zentromart-Voiceops

Project Documentation: README.md

Setup Instructions: SETUP.md

Demonstration Guide: DEMO.md

10. References and Acknowledgements

The project was developed using the VoiceOps 2026 capstone brief and the official synthetic ZentroMart dataset and mock tools provided by the organisers and C&A / Gnani.ai.

Technical references include the Gnani Python SDK documentation for speech-to-text and text-to-speech and the Groq API documentation.

AI assistance was used during parts of implementation, debugging and report preparation. The reported evaluation results are based on saved project outputs, and the offline test results are based on the executed test suite.